

PokerBots

Environment setup, Kuhn Poker, baseline bots, and evaluation

Ravpreet Singh Ahluwalia

Week 1

Week 1 motivation: why start here?

Goal

Before implementing CFR/MCCFR, we first needed a reliable experimental pipeline.

What we built

- Windows Python virtual environment.
- OpenSpiel-based Kuhn Poker setup.
- Five baseline bots.
- Round-robin tournament runner.
- CSV results and report generation.

Why it matters

- Establishes reproducibility.
- Catches bugs before CFR.
- Gives baseline performance levels.
- Creates a pipeline: simple bots $\rightarrow$ regret minimization.

What OpenSpiel gives us vs. what we wrote

OpenSpiel gives us

- `kuhn_poker` game rules.
- Legal actions at each state.
- Chance outcomes and probabilities.
- State transitions via `apply_action`.
- Terminal payoffs via `returns()`.

We wrote

- Baseline bot policies.
- Duplicate-pair evaluation.
- Round-robin tournament loop.
- Statistics and CSV output.
- Observer transcript script.

OpenSpiel is the referee/table; our code supplies agents and experiments.

Kuhn Poker: compact game description

Rules

- Three-card deck: Jack, Queen, King.
- Each player antes 1 chip.
- Each player receives one private card.
- One betting round: pass/check/fold or bet/call.
- Highest private card wins at showdown.

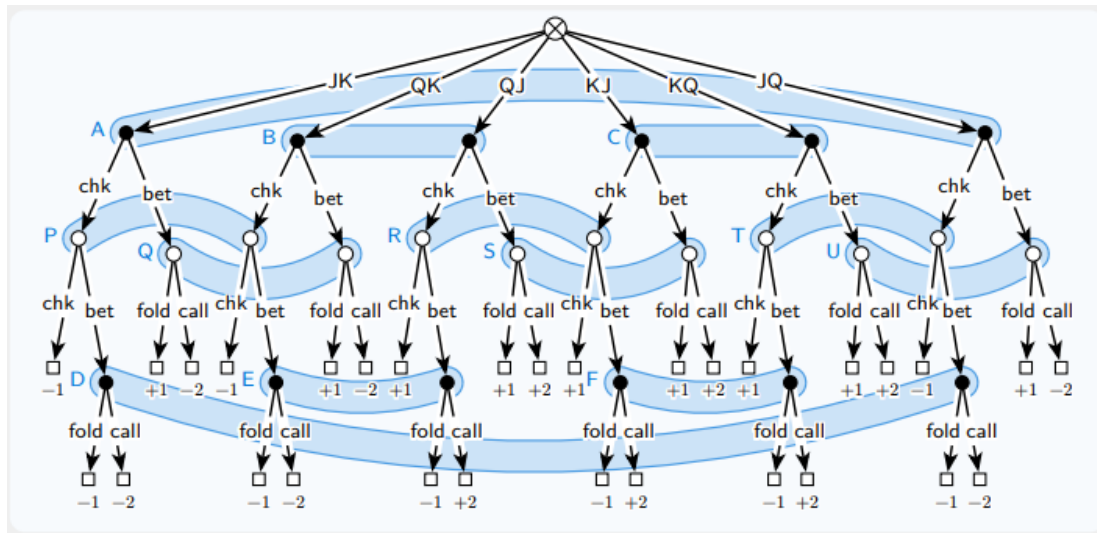
OpenSpiel encodings

Card id	Card
0	Jack
1	Queen
2	King

Action id	Meaning
0	p: pass/fold
1	b: bet/call

The action meaning depends on context: p can be pass or fold; b can be bet or call.

Kuhn Poker information structure



Baseline 1: Random Agent

Rule

Uniformly sample one action from `state.legal_actions()`.

```
legal = state.legal_actions()
return int(self.rng.choice(legal))
```

Why choose it?

- It is the lowest nontrivial baseline.
- It tests whether the game loop and legal-action interface work.
- Stronger Agent must always beat Random Play.

Baseline 2: Always-Call

Rule

Choose action 1 whenever legal; otherwise choose a legal fallback.

```
legal = state.legal_actions()
return 1 if 1 in legal else int(legal[0])
```

Why choose it?

- It is a simple over-aggressive fixed strategy.
- It is predictable and exploitable.
- It checks whether stronger bots can punish aggressive play.

Baseline 3: Always-Fold

Rule

Choose action 0 whenever legal; otherwise choose a legal fallback.

```
legal = state.legal_actions()  
return 0 if 0 in legal else int(legal[0])
```

Why choose it?

- It is a simple passive degenerate strategy.
- It exposes whether stronger bots can exploit passive play well.

Domain-knowledge bot: Rule-Based Agent

Rule after update

Use private-card strength and whether the player is facing a bet.

Card	No bet pending	Facing bet
Jack	check	fold/check
Queen	check	call
King	bet	call

Why choose it?

- First bot with poker domain knowledge.
- Still simple and interpretable.
- Serves as a bridge between degenerate bots and EV/CFR logic.

EV/randomized heuristic bot

Core idea

A slightly more principled heuristic: strong cards play aggressively, weak cards play passively, and middle cards mix.

```
if card == 1: # Queen
    return aggressive if self.rng.random() < 0.35 else passive
```

Why choose it?

- Introduces expected-payoff intuition before CFR.
- Uses stochastic behavior for the middle card.
- Gives a stronger baseline that rule-based logic may not beat.

The 0.35 is hand-chosen, not theoretically optimal. CFR later learns these probabilities from regret minimization.

Evaluation protocol

Round-robin

- 5 bots create $\binom{5}{2} = 10$ matchups.
- Each matchup is run across seeds 0 to 4.
- Results are saved to CSV.

Duplicate-pair evaluation

- Same card-deal seed is reused twice.
- Hand 1: A as Player 0, B as Player 1.
- Hand 2: B as Player 0, A as Player 1.
- Average payoff to A across both seats.

Why duplicate pairs?

They reduce card-luck and seat-position variance, so the comparison is more about strategy than lucky deals.

Final Week 1 results: summary

game	agent_a	agent_b	mean_pay	avg_ci_lo	avg_ci_hig	avg_win_r	avg_std
kuhn_poke	always_ca	always_fo	1	1	1	1	0
kuhn_poke	random	always_fo	0.50104	0.49124	0.51084	0.50104	0.500001
kuhn_poke	always_fo	rule_based	0	0	0	0	0
kuhn_poke	rule_based	ev_heurist	-0.08964	-0.0934	-0.08588	0	0.191794
kuhn_poke	always_fo	ev_heurist	-0.11668	-0.12297	-0.11039	0	0.321033
kuhn_poke	always_ca	ev_heurist	-0.15225	-0.1634	-0.1411	0.18042	0.568846
kuhn_poke	random	ev_heurist	-0.16136	-0.17136	-0.15136	0.12546	0.510389
kuhn_poke	random	rule_based	-0.21112	-0.21705	-0.20519	0.0612	0.302633
kuhn_poke	always_ca	rule_based	-0.33396	-0.33858	-0.32934	0	0.235483
kuhn_poke	random	always_ca	-0.38203	-0.39738	-0.36668	0.24846	0.782978

Positive means Agent A beats Agent B; negative means Agent A loses to Agent B. Mean payoff is the main metric because poker payoffs are not equal-sized wins/losses.

Interpreting the results

- **Rule-based beats always-call:** `always_call` vs `rule_based` = -0.334. The rule-based bot punishes over-aggression.
- **EV heuristic beats rule-based:** `rule_based` vs `ev_heuristic` = -0.0896. This is consistent with EV being the strongest Week 1 heuristic baseline.
- **Always-fold vs rule-based is exactly 0:** duplicate-pair averaging cancels the seat/card advantage in this simple matchup.
- **Random loses to stronger baselines:** random loses to EV, rule-based, and always-call.

Main conclusion

The infrastructure works: bots choose legal actions, duplicate matches run, metrics are recorded, and the baseline ladder behaves plausibly.

What we will do next week

Week 2: regret matching

Move from fixed heuristics to no-regret learning.

- Implement regret matching from scratch on Rock-Paper-Scissors.
- Track average strategy convergence toward Nash.
- Plot average regret R_T/T over iterations.
- Explain why local regret minimization is the conceptual bridge to CFR.
- Prepare for vanilla CFR on Kuhn Poker in Week 3.

Thank you